

Creacion de repositorio con supabase Taller.

Todo está adaptado para la implementación de **[Nombre de tu Aplicativo]**.

0. Requisitos Previos en tu Equipo Local

Dado que no usarás entornos en la nube como Codespaces, asegúrate de tener instalado en tu computadora:

- **Node.js:** Versión 22.22 o superior.
- **Git:** Para el control de versiones local.
- **PostgreSQL local:** Necesario para correr las pruebas automáticas en tu máquina (puedes instalarlo descargándolo de su página oficial según tu sistema operativo).
- **Cuenta de GitHub:** Para crear el repositorio remoto al final.
- **Cuenta de Supabase:** Plan gratuito en supabase.com.

1. Preparación y Base de Datos (Supabase)

1. **Crear el proyecto en Supabase:** En supabase.com, selecciona "New project". Nómbralo `[nombre-de-tu-proyecto]`, genera una contraseña segura (guárdala en un gestor de contraseñas de tu equipo) y selecciona la región más cercana.
2. **Obtener credenciales:** Ve a *Project Settings* → *API Keys*. Copia la **Project URL** y la **Clave publicable** (`sb_publishable_...`). Nunca expongas la **Clave secreta** (`sb_secret_...`).
3. **Configurar Autenticación:** En *Authentication* → *URL Configuration*, establece tu *Site URL* (ej. `https://[tu-usuario].github.io/[nombre-de-tu-proyecto]/`) y añade `http://localhost:5173/**` en *Redirect URLs* para tu desarrollo local.
4. **Vincular el proyecto por CLI (Localmente):** Abre la terminal en la carpeta local de tu proyecto y ejecuta:

- `npx supabase login` (Esto abrirá automáticamente el navegador de tu computadora para que autorices)
 - `npx supabase init`
 - `npx supabase link --project-ref [referencia-de-tu-proyecto]` (Te pedirá la contraseña del paso 1)
5. **Crear y aplicar migraciones:** Usa `npx supabase migration new [nombre_migracion]` para crear archivos SQL. Aquí definirás:
- **Esquema:** Tablas principales de **[Nombre de tu Aplicativo]**.
 - **Reglas y Seguridad:** Disparadores (triggers) y políticas RLS (Row Level Security) para proteger los datos.
 - **Funciones:** Procedimientos para lógica de negocio.
6. **Subir los cambios:** Ejecuta `npx supabase db push` para subir tus migraciones locales a la base de datos en la nube. Carga tus datos de prueba usando un archivo `seed.sql` directamente en el SQL Editor de Supabase.

2. Configuración del Frontend (React en Local)

1. **Inicializar el proyecto:** En la raíz de tu carpeta local, ejecuta:
- `npm create vite@latest frontend -- --template react`
 - `cd frontend && npm install`
 - `npm install react-router @supabase/supabase-js`
2. **Variables de entorno:** Crea un archivo `.env.local` y asegúrate de que no se subirá al repositorio (verifica que esté en el archivo `.gitignore`). Incluye:
- `VITE_SUPABASE_URL=https://[referencia].supabase.co`
 - `VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_...`
3. **Organización del código:**
- Mantén la conexión a Supabase centralizada en un solo archivo (ej. `lib/supabase.js`).
 - Traduce los errores de PostgreSQL a mensajes amigables para el usuario.

- Recuerda: la validación en React es visual; la seguridad real de **[Nombre de tu Aplicativo]** está en las reglas de tu base de datos.
4. **Ejecución y pruebas locales:** Corre `npm run dev`. Prueba todo el flujo (registro, creación de registros, cancelaciones, límites) directamente en `http://localhost:5173`.

3. Control de Versiones Local y Despliegue (GitHub Pages)

Dado que empezaste en local, aquí se detalla cómo pasar tus cambios al entorno remoto para publicar.

1. Inicializar y guardar en Git local:

En la raíz de tu proyecto, abre tu terminal y ejecuta:

- `git init` (Si no lo has hecho aún)
- `git add .`
- `git status` (REVISIÓN CRÍTICA: Asegúrate de que `.env.local` NO aparezca en verde. Si aparece, arréglalo en tu `.gitignore`)
- `git commit -m "Configuración inicial de [Nombre de tu Aplicativo] y migraciones"`

2. Configurar el Workflow de despliegue: Antes de subirlo, configura GitHub Actions. En tu archivo `.github/workflows/desplegar-frontend.yml` asegúrate de:

- Incluir `BASE_PATH: /${{ github.event.repository.name }}/`
- Agregar el paso para el enrutamiento: `cp dist/index.html dist/404.html`

3. Crear el repositorio remoto: Ve a GitHub.com, crea un repositorio vacío llamado `[nombre-de-tu-proyecto]` (sin README ni `.gitignore` adicionales).

4. Vincular y subir (Push): Copia los comandos que te da GitHub para repositorios existentes, que normalmente son:

- `git remote add origin [https://github.com/](https://github.com/)[tu-usuario]/[nombre-de-tu-proyecto].git`
- `git branch -M main`
- `git push -u origin main`

5. **Configurar las Variables en GitHub:** Ve a tu repositorio remoto en *Settings* → *Secrets and variables* → *Actions* → *Variables* y agrega `VITE_SUPABASE_URL` y `VITE_SUPABASE_PUBLISHABLE_KEY`.
6. **Activar GitHub Pages:** Ve a *Settings* → *Pages* → *Source* y selecciona **GitHub Actions**. El workflow compilará tu aplicación de Vite automáticamente.
7. **Verificación final:** Abre la URL pública generada por GitHub Pages. Comprueba que el inicio de sesión y la lectura de datos de Supabase funcionen correctamente en el entorno de producción.